Department of Computer Science and Engineering

FOOD DELIVERY MANAGEMENT SYSTEM

MICRO PROJECT REPORT

Submitted by

NIDA PARVEEN

ROSMI BABU

NANDANA T S

SANA NASRIN

B.Tech. CSE

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelors of Technology

In

Computer Science and Engineering

March 2026

ABSTRACT

The Food Delivery Management System is a desktop-based application developed using the Python programming language. The main purpose of this system is to help manage customer details, food items, and food orders in an organized and efficient manner. Many small food businesses require a simple system to handle their daily operations, and this application is designed to provide a basic solution for managing food delivery activities. By using this system, users can easily maintain records and reduce the difficulties associated with manual record keeping.

The graphical user interface of the system is developed using the Tkinter library in Python. Tkinter provides various interface components such as buttons, labels, text fields, and forms that allow users to interact with the application easily. Through this interface, users can enter customer details, add food items with their prices, and create food orders. The GUI design makes the system simple to operate and allows even non-technical users to perform the required tasks without difficulty.

MongoDB is used as the database for storing the system data. It is a NoSQL database that stores information in the form of collections and documents instead of traditional tables. In this project, MongoDB is used to store customer details, food item information, and order data in a structured way. This ensures that the information remains organized and can be easily retrieved, updated, or managed whenever needed.

The system allows users to add new customers, maintain food item records, and create food orders in an efficient manner. When a user creates an order, the system automatically calculates the total price based on the selected food item and the quantity entered. This automatic calculation feature helps reduce manual errors and ensures that order details are accurate.

Overall, this project demonstrates how a Python GUI application can be integrated with a database to create a simple and practical food delivery management system. It shows the effective use of Python programming, graphical user interface development with Tkinter, and database management using MongoDB to build a functional application that can assist in managing basic food delivery operations.

INDEX

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

Food delivery services are widely used today, and restaurants require efficient systems to manage customers, food items, and orders. However, many small restaurants still use manual methods like notebooks, which can lead to errors and difficulty in maintaining records. To solve this problem, the Food Delivery Management System was developed using Python Tkinter for the graphical user interface and MongoDB for data storage. The system helps users manage customer details, food items, and orders easily. It also automatically calculates the total price based on quantity, reducing calculation errors. The application provides a simple and user-friendly solution for managing food delivery operations.

1.1 Objective

The main objective of this project is to develop a simple Food Delivery Management System to manage food orders efficiently. The system provides a user-friendly GUI using Python Tkinter and stores data using MongoDB. It helps manage customer details, food items with prices, and allows easy order creation. The system also automatically calculates the total price of each order, reducing errors and improving efficiency.

1.2 Problem Statement

Many small food businesses manage orders manually, which can cause errors, data loss, and difficulty in maintaining records. Manual systems are also time-consuming and inefficient when the number of orders increases. Therefore, a computerized system is needed to manage customers, food items, and orders efficiently. The Food Delivery Management System solves these problems by using a database and a simple graphical interface to improve accuracy and organization.

1.3 Scope

The scope of this project is to manage food delivery operations using a simple desktop application. The system allows users to manage customer details, food items, and create orders easily. It automatically calculates order prices and stores all data in a MongoDB database for easy access. This project is mainly for educational purposes and is suitable for small-scale food delivery services.

CHAPTER 2

MODULAR DESCRIPTION

The Food Delivery Management System has three main modules. These modules help the system manage customer details, food items, and food orders in an organized way.

2.1 Customer Management Module

The Customer Management Module is used to manage customer information in the system. It allows the user to add new customers and store their details in the database. The user can also update customer details when needed. If a customer record is not required, it can be deleted from the system. This module also allows the user to view all customer records stored in the database. The customer information stored in the system includes Customer ID, Customer Name, Phone Number, and Address.

2.2 Food Item Management Module

The Food Item Management Module is used to manage the food items available in the system. It allows the user to add new food items and their prices. If any changes are required, the user can update the food item details. The system also allows deletion of food items that are no longer available. Users can also view all the food items stored in the database. The food item details stored include Food ID, Food Name, and Price.

2.3 Order Management Module

The Order Management Module is used to manage customer orders. In this module, the user can create a new order by selecting a customer and a food item. The user can enter the quantity of the food item ordered. The system automatically calculates the total price based on the price and quantity. This helps avoid manual calculation errors. The order details stored in the system include Order ID, Customer ID, Food ID, Quantity, and Total Price.

CHAPTER 3

FRONT END & BACK END

3.1 Python

Python is the main programming language used to develop the system. It handles the application logic and processes user inputs.

3.2MongoDB

MongoDB is a NoSQL database used to store the data required for the system. Unlike traditional databases that use tables, MongoDB stores information in the form of collections and documents, which makes data storage more flexible and easier to manage. In this system, MongoDB is used to store different types of information, including customer data, food item data, and order data. This allows the application to organize and retrieve information efficiently whenever it is needed.

3.3 Tkinter

Tkinter is a Python library used to create graphical user interfaces (GUI) for desktop applications. It provides a variety of components such as buttons, labels, text boxes, and other interface elements that help in designing an interactive and user-friendly application interface. With the help of Tkinter, developers can create windows, forms, and input fields that allow users to interact easily with the system. In this project, Tkinter is used to design the application interface where users can enter customer details, manage food items, and create food orders. This makes the system simple to use and improves the overall user experience.

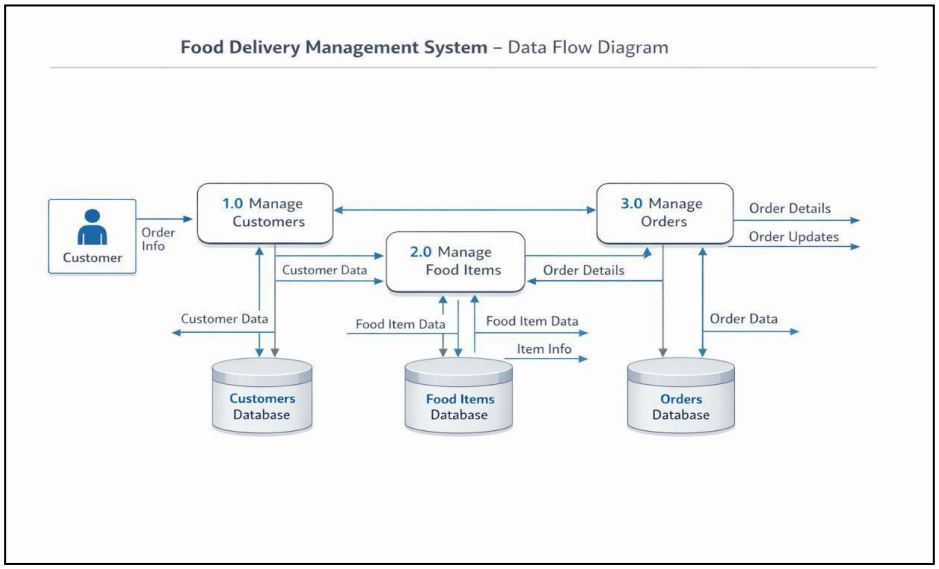
3.4 PyMongo

PyMongo is a Python library that allows Python applications to connect and interact with a MongoDB database. It acts as a bridge between the Python program and the MongoDB server, enabling the application to communicate with the database easily. Using PyMongo, the system can perform various database operations such as inserting new records, updating existing records, deleting unwanted data, and retrieving stored information when required. This library makes it possible for the application to manage and manipulate data efficiently within the MongoDB database, ensuring that customer details, food items, and order information can be stored and accessed smoothly.

CHAPTER 4

SYSTEM DESIGN

4.1 Data Flow Diagram (DFD)



This diagram shows how user input is processed and stored in different database collections.

4.2 Tabular design

Food Items Table

Field	Data Type	Description
food_id	String	Unique ID for each food item
food_name	String	Name of the food item
price	Float	Price of the food item

Fig 4.2.1. Food items

Customer Table

Field	Data Type	Description
customer_id	String	Unique ID assigned to each customer
name	String	Name of the customer
phone	String	Contact number of the customer
address	String	Delivery address of the customer

Fig 4.2.1 Customer items

Fig 4.2.2 Order items

Orders Table

Field	Data Type	Description
order_id	String	Unique ID assigned to each order
customer_id	String	ID of the customer who placed the order
food_id	String	ID of the ordered food item
quantity	Integer	Number of food items ordered
total_price	Float	Total cost of the order
status	String	Status of the order (Pending, Preparing, Delivered)

CHAPTER 5

TESTING AND DESIGN

5.1 Source Code

```
import tkinter as tk

from tkinter import messagebox, ttk

from pymongo import MongoClient, errors
```

DATABASE CONNECTION

```
client = MongoClient("mongodb://localhost:27017/")

db = client["food_delivery_db"]

customers = db["customers"]

foods = db["food_items"]

orders = db["orders"]

customers.create_index("customer_id", unique=True)

foods.create_index("food_id", unique=True)

orders.create_index("order_id", unique=True)
```

MAIN WINDOW

```
root = tk.Tk()

root.title("Food Delivery Management System")

root.geometry("1100x600")

root.configure(bg="#ffe5b4")
```

TITLE

```
title = tk.Label(root,

text=" Food Delivery Management System",

font=("Helvetica",20,"bold"),
```

```
bg="#ff914d",

fg="white",

pady=10)

title.grid(row=0,column=0,columnspan=3,sticky="ew")
```

CUSTOMER SECTION

```
 customer_frame = tk.LabelFrame(root,text="Customer Section",

font=("Arial",12,"bold"),bg="#fff4e6",padx=10,pady=10)

customer_frame.grid(row=1,column=0,padx=20,pady=20)

tk.Label(customer_frame,text="Customer ID",bg="#fff4e6").grid(row=0,column=0)

cust_id = tk.Entry(customer_frame)

cust_id.grid(row=0,column=1)

tk.Label(customer_frame,text="Name",bg="#fff4e6").grid(row=1,column=0)

cust_name = tk.Entry(customer_frame)

cust_name.grid(row=1,column=1)

tk.Label(customer_frame,text="Phone",bg="#fff4e6").grid(row=2,column=0)

cust_phone = tk.Entry(customer_frame)

cust_phone.grid(row=2,column=1)

tk.Label(customer_frame,text="Address",bg="#fff4e6").grid(row=3,column=0)

cust_address = tk.Entry(customer_frame)

cust_address.grid(row=3,column=1)
```

CLEAR CUSTOMER

```
def clear_customer():

cust_id.delete(0,tk.END)

cust_name.delete(0,tk.END)

cust_phone.delete(0,tk.END)

cust_address.delete(0,tk.END)
```

CUSTOMER FUNCTIONS

```
def add_customer():
try:
customers.insert_one({
"customer_id":cust_id.get(),
"name":cust_name.get(),
"phone":cust_phone.get(),
"address":cust_address.get() })
messagebox.showinfo("Success","Customer Added!")
clear_customer()
except errors.DuplicateKeyError:
messagebox.showerror("Error","Customer ID already exists!")
def update_customer():
customers.update_one(
{"customer_id":cust_id.get()},
{"$set":{
"name":cust_name.get(),
"phone":cust_phone.get(),
"address":cust_address.get() }})
messagebox.showinfo("Updated","Customer Updated")
def delete_customer():
customers.delete_one({"customer_id":cust_id.get()})
messagebox.showinfo("Deleted","Customer Deleted")
clear_customer()
def view_customers():
window=tk.Toplevel(root)
```

```
window.title("Customer List")

tree=ttk.Treeview(window,

columns=("ID","Name","Phone","Address"),

show="headings")

tree.heading("ID",text="Customer ID")

tree.heading("Name",text="Name")

tree.heading("Phone",text="Phone")

tree.heading("Address",text="Address")

tree.pack(fill="both",expand=True)

for c in customers.find():

tree.insert("", "end",

values=(c["customer_id"],c["name"],c["phone"],c["address"]))

tk.Button(customer_frame,text="Add
Customer",bg="#ff914d",fg="white",command=add_customer).grid(row=4,column=0,columnspan=2,
pady=5)

tk.Button(customer_frame,text="Update
Customer",bg="#3498db",fg="white",command=update_customer).grid(row=5,column=0,columnspa
n=2,pady=5)

tk.Button(customer_frame,text="Delete
Customer",bg="#e74c3c",fg="white",command=delete_customer).grid(row=6,column=0,columnspan
=2,pady=5)

tk.Button(customer_frame,text="View
Customers",bg="#f39c12",fg="white",command=view_customers).grid(row=7,column=0,columnspa
n=2,pady=5)
```

FOOD SECTION

```
food_frame = tk.LabelFrame(root,text="Food Section",

font=("Arial",12,"bold"),bg="#e6fff2",padx=10,pady=10)

food_frame.grid(row=1,column=1,padx=20,pady=20)
```

```
tk.Label(food_frame,text="Food ID",bg="#e6fff2").grid(row=0,column=0)
food_id=tk.Entry(food_frame)
food_id.grid(row=0,column=1)
tk.Label(food_frame,text="Food Name",bg="#e6fff2").grid(row=1,column=0)
food_name=tk.Entry(food_frame)
food_name.grid(row=1,column=1)
tk.Label(food_frame,text="Price",bg="#e6fff2").grid(row=2,column=0)
food_price=tk.Entry(food_frame)
food_price.grid(row=2,column=1)
def clear_food():
food_id.delete(0,tk.END)
food_name.delete(0,tk.END)
food_price.delete(0,tk.END)
def add_food():
try:
foods.insert_one({
"food_id":food_id.get(),
"food_name":food_name.get(),
"price":float(food_price.get()) })
messagebox.showinfo("Success","Food Added!")
clear_food()
except errors.DuplicateKeyError:
messagebox.showerror("Error","Food ID already exists!")
def update_food():
foods.update_one(
{"food_id":food_id.get()},
```

```
{"$set":{
"food_name":food_name.get(),
"price":float(food_price.get())}})
messagebox.showinfo("Updated","Food Updated")
def delete_food():
foods.delete_one({"food_id":food_id.get()})
messagebox.showinfo("Deleted","Food Deleted")
clear_food()
def view_food():
window=tk.Toplevel(root)
window.title("Food Items")
tree=ttk.Treeview(window,
columns=("ID","Name","Price"),
show="headings")
tree.heading("ID",text="Food ID")
tree.heading("Name",text="Food Name")
tree.heading("Price",text="Price")
tree.pack(fill="both",expand=True)
for f in foods.find():
tree.insert("", "end",
values=(f["food_id"],f["food_name"],f["price"]))
tk.Button(food_frame,text="Add
Food",bg="#00b894",fg="white",command=add_food).grid(row=3,column=0,columnspan=2,pady=5)
tk.Button(food_frame,text="Update
Food",bg="#00cec9",fg="white",command=update_food).grid(row=4,column=0,columnspan=2,pady
=5)
```

```
tk.Button(food_frame,text="Delete
Food",bg="#d63031",fg="white",command=delete_food).grid(row=5,column=0,columnspan=2,pady
=5)

tk.Button(food_frame,text="View
Food",bg="#27ae60",fg="white",command=view_food).grid(row=6,column=0,columnspan=2,pady=
5)
```

ORDER SECTION

```
order_frame = tk.LabelFrame(root,text="Order Section",

font=("Arial",12,"bold"),bg="#e6f2ff",padx=10,pady=10)

order_frame.grid(row=1,column=2,padx=20,pady=20)

tk.Label(order_frame,text="Order ID",bg="#e6f2ff").grid(row=0,column=0)

order_id=tk.Entry(order_frame)

order_id.grid(row=0,column=1)

tk.Label(order_frame,text="Customer ID",bg="#e6f2ff").grid(row=1,column=0)

order_customer=tk.Entry(order_frame)

order_customer.grid(row=1,column=1)

tk.Label(order_frame,text="Food ID",bg="#e6f2ff").grid(row=2,column=0)

order_food=tk.Entry(order_frame)

order_food.grid(row=2,column=1)

tk.Label(order_frame,text="Quantity",bg="#e6f2ff").grid(row=3,column=0)

order_quantity=tk.Entry(order_frame)

order_quantity.grid(row=3,column=1)

def clear_order():

order_id.delete(0,tk.END)

order_customer.delete(0,tk.END)

order_food.delete(0,tk.END)

order_quantity.delete(0,tk.END)
```

```
def create_order():
try:
food=foods.find_one({"food_id":order_food.get()})
if food:
total=int(order_quantity.get())*food["price"]
orders.insert_one({
"order_id":order_id.get(),
"customer_id":order_customer.get(),
"food_id":order_food.get(),
"quantity":int(order_quantity.get()),
"total_price":total,
"status":"Pending" })
messagebox.showinfo("Success","Order Created!")
clear_order()
else:
messagebox.showerror("Error","Food not found")
except errors.DuplicateKeyError:
messagebox.showerror("Error","Order ID exists")
def delete_order():
orders.delete_one({"order_id":order_id.get()})
messagebox.showinfo("Deleted","Order Deleted")
clear_order()
def view_orders():
window=tk.Toplevel(root)
window.title("Orders")
    tree=ttk.Treeview(window,
```

```
    columns=("OrderID","CustomerID","FoodID","Qty","Total","Status"),
    show="headings")
    tree.heading("OrderID",text="Order ID")
    tree.heading("CustomerID",text="Customer ID")
    tree.heading("FoodID",text="Food ID")
    tree.heading("Qty",text="Quantity")
    tree.heading("Total",text="Total")
    tree.heading("Status",text="Status")
    tree.pack(fill="both",expand=True)
    for o in orders.find():
        tree.insert("", "end",
        values=(o["order_id"],o["customer_id"],o["food_id"],
        o["quantity"],o["total_price"],o["status"]))
tk.Button(order_frame,text="Create
Order",bg="#0984e3",fg="white",command=create_order).grid(row=4,column=0,columnspan=2,pad
y=5)
tk.Button(order_frame,text="Delete
Order",bg="#c0392b",fg="white",command=delete_order).grid(row=5,column=0,columnspan=2,pad
y=5)
tk.Button(order_frame,text="View
Orders",bg="#6c5ce7",fg="white",command=view_orders).grid(row=6,column=0,columnspan=2,pad
y=5)
root.mainloop()
```

RESULT AND ANALYSIS

The Food Delivery Management System was successfully developed and tested. The system was able to store customer details, food items, and order information efficiently in the MongoDB database. Through the graphical user interface created using Tkinter, users were able to add, view, and manage data in a simple and organized manner. The interface allowed users to enter customer information, maintain details of available food items, and create food orders without difficulty.

The system performed all its main operations correctly, including adding new customers, adding and updating food items, and creating orders. When an order was created, the system automatically calculated the total price based on the selected food items and their quantities. This automatic price calculation feature helped reduce manual calculation errors and improved the accuracy of order processing.

The graphical user interface played an important role in making the system easy to use. Users could easily navigate between different sections of the application, such as customer management, food item management, and order creation. The layout and interface components provided a clear and simple way for users to perform the required operations.

The connection between the Python application and the MongoDB database was successfully established using the PyMongo library. This allowed the system to store, retrieve, update, and manage data efficiently within the database. Overall, the system worked effectively and demonstrated how a simple database-driven application can be developed using Python, Tkinter, and MongoDB to manage food delivery operations in an organized and efficient manner.

CONCLUSION

The Food Delivery Management System was successfully designed and implemented using Python Tkinter and MongoDB. The main aim of this project was to develop a simple and user-friendly application to manage customer details, food items, and food orders efficiently. The system provides a graphical user interface using Tkinter, which makes it easy for users to interact with the application without requiring advanced technical knowledge.

Through this system, users can add and manage customer information, maintain food item records, and create orders in an organized way. The application automatically calculates the total price based on the selected food item and quantity, which helps reduce calculation errors and save time. MongoDB is used as the database to store customer, food, and order details in a structured and easily accessible manner.

The project also shows how a Python GUI application can be connected with a database using the PyMongo library. It demonstrates important concepts such as database management, GUI development, and application programming. The system performs all required operations effectively and provides a simple solution for managing food delivery tasks.

The system can be further improved by adding features like online ordering, payment integration, order tracking, and delivery management. Improvements in user interface design and data security can also enhance the system. This project serves as a good base for developing more advanced food delivery applications in the future.

APPENDICES

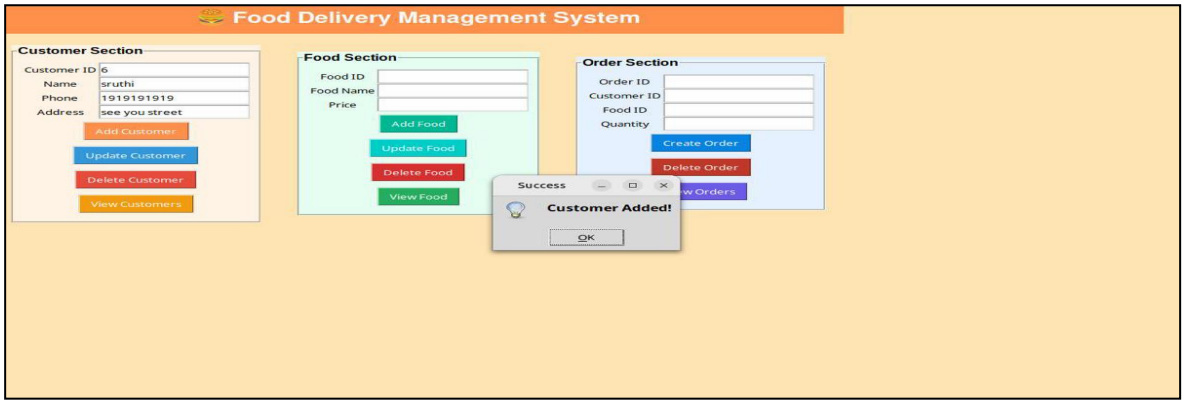


Fig 5.1 Customer added

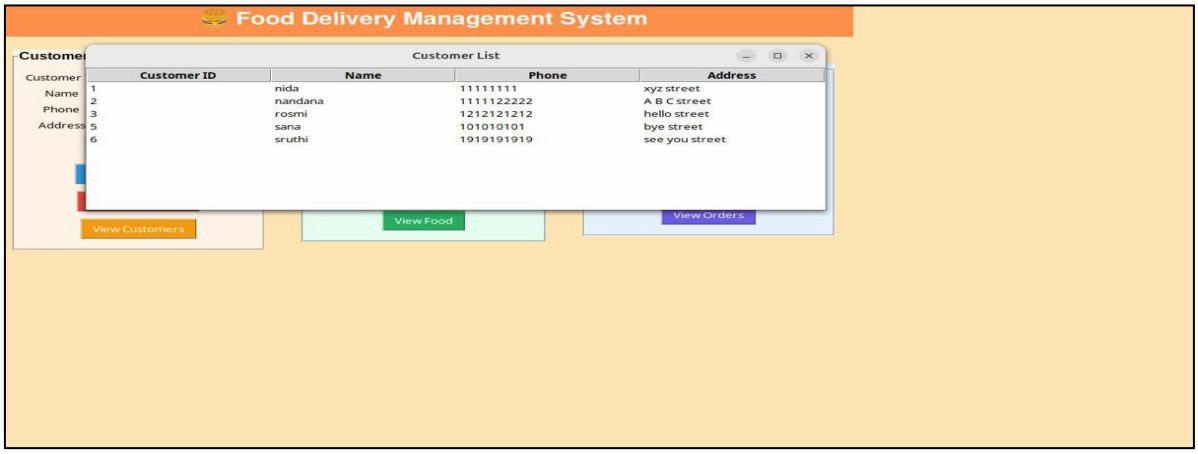


Fig 5.2 Customer Added view

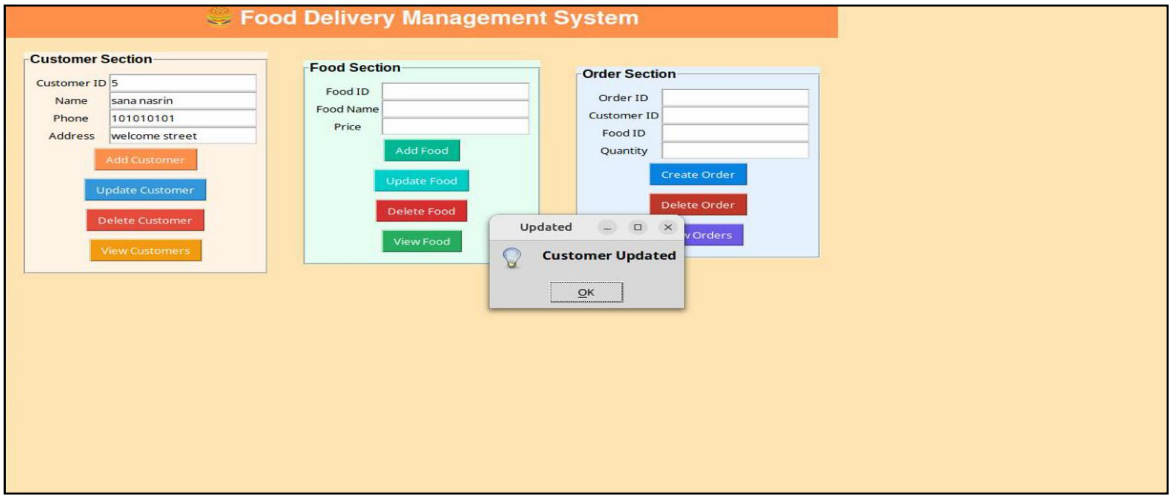


Fig 5.3 Customer updated

Fig 5.4 Food updated

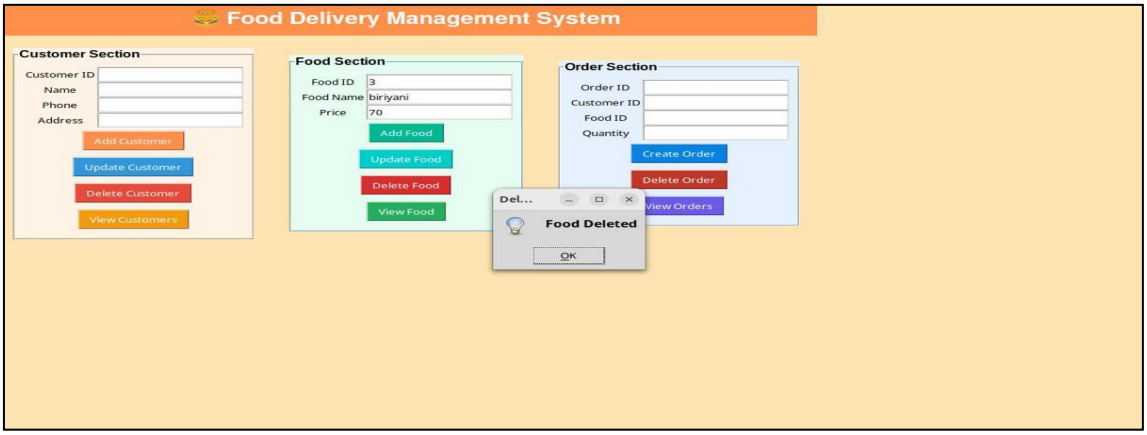


Fig 5.5 Food deleted

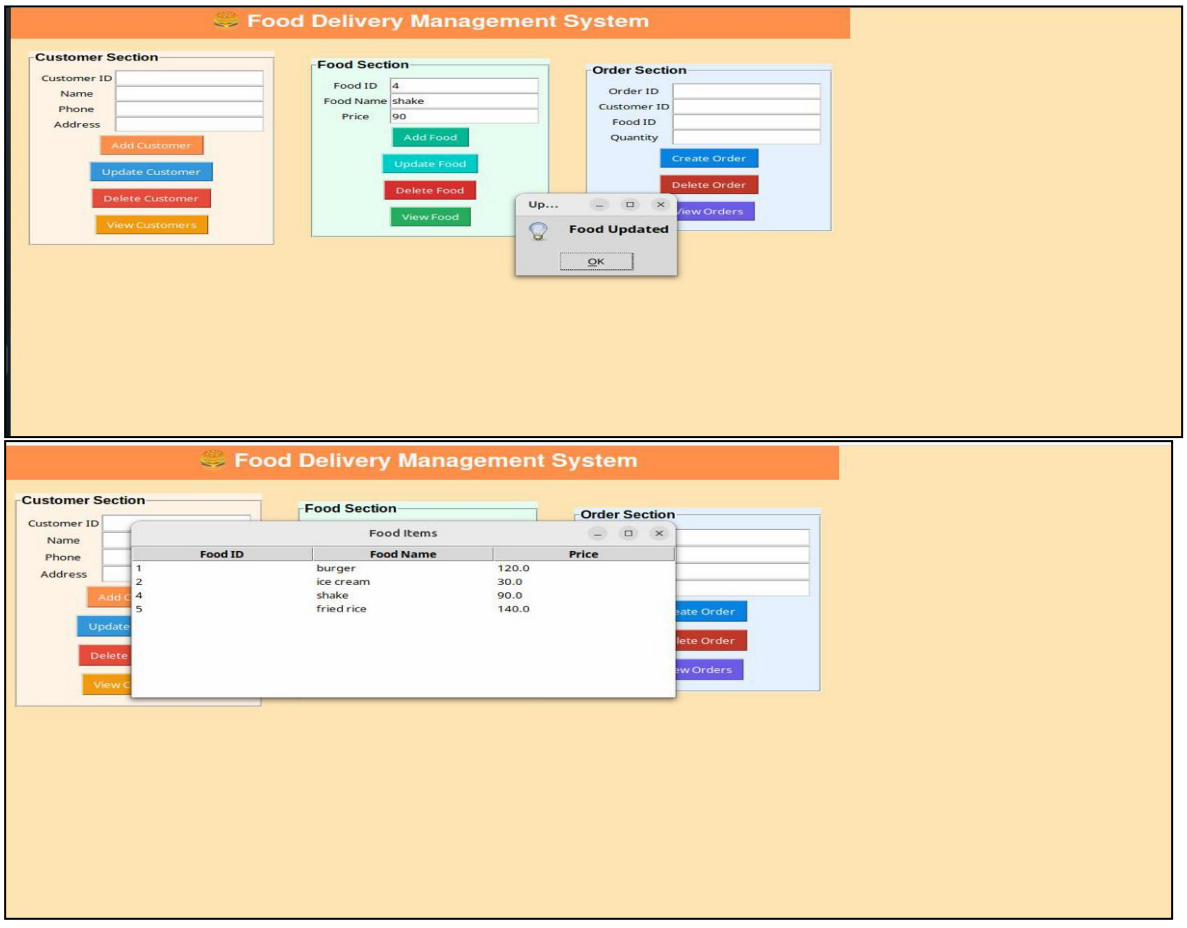


Fig 5.6 Food deleted view

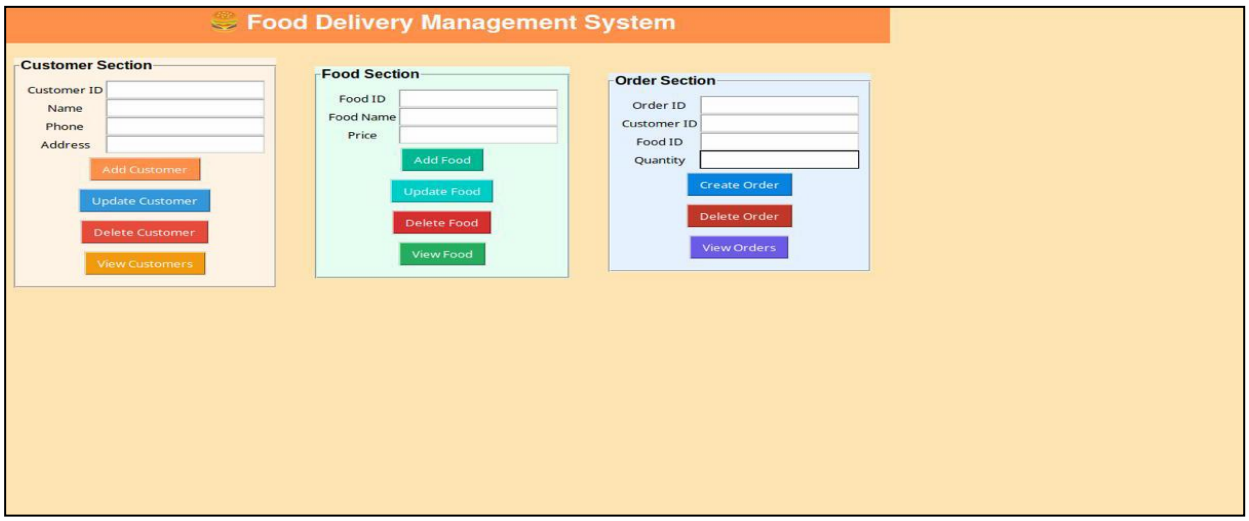


Fig 5.7 Interface

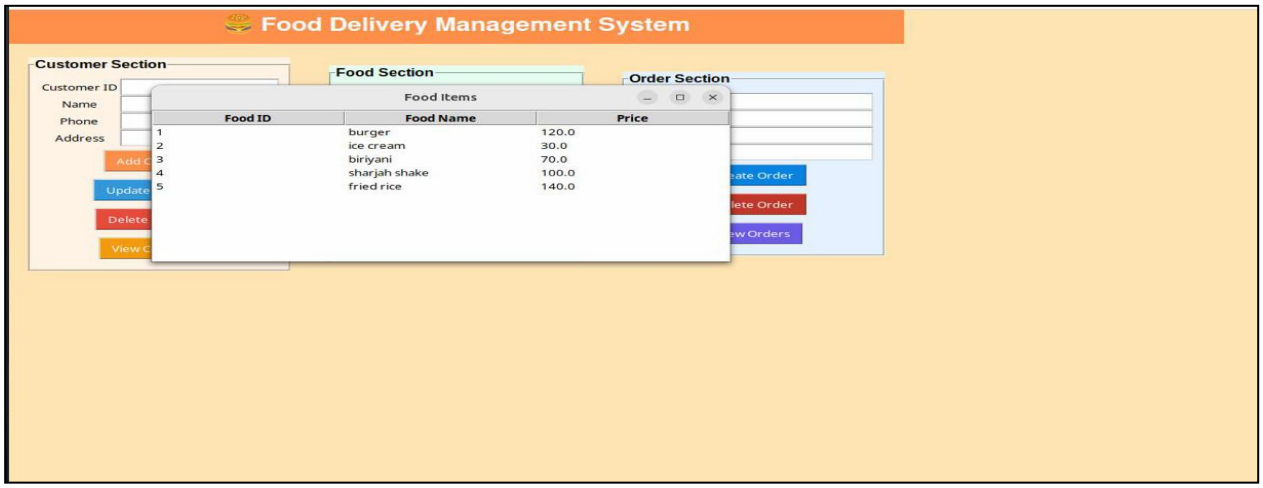


Fig 5.8 Food added